

Lean, Not Forgetful

A measured verdict first: one line in three is commentary, and almost none of it is noise — it is verbose signal. So: the one-line comment standard for all new code, compression of the two dozen hot files where the token cost actually lives, a sacred list that keeps every scar, an evidence-led deletion table, and the practice fixes — one sign-in helper instead of twenty, the VRT diff tool the law already assumes, guard plumbing shared without moving a single assertion.

“Strip the noise, keep the scars.”

— the organizing rule

The repo sheds weight without shedding memory. Plan B of two (its sibling is CLARITY-AND-CADENCE-PLAN.md). Three phases, `lc-1` ... `lc-3`. Written 2026-07-15 from a measured audit of every tracked file: comment density per directory, reference counts for every deletion candidate, and a sweep of the build/test practice for disproportionate token and time costs. Executor: Claude Opus 4.8, one phase per session, under CLAUDE.md's standing rules.

Who this document speaks to. Executed by Claude Opus 4.8; the imperatives address that executor. Part 0's rows and Part 1's argument ("what you asked for", "your instinct") address **Bishan**, the commissioning user — the executor never answers or waits on them; silence means the stated default. One deliberate exception: the blockquoted CLAUDE.md amendment in Part 1 is the standard's own text, and its "you" correctly addresses whichever session edits a file in future.

THE AUTONOMY CONTRACT (binding; restates the standing directive of 2026-07-11). One phase per session; within a phase never ask, never wait. Questions become QUESTIONS-FOR-BISHANT.md entries with marked assumptions; Part 0's defaults proceed unanswered; only an unworkaroudable blocker stops work. Session ritual per CLAUDE.md at both ends of every session.

Sequencing. Runs AFTER Plan A's CC1 (the two live product defects do not wait on hygiene) and BEFORE CC2–CC10 (so the new comment standard governs nine sessions of new code, the hot files those sessions will read repeatedly are already lean, and this plan's wide mechanical touches never churn files Plan A just rewrote). Within this plan: the standard first, consolidation second, compression last. **Deletions and wide touches happen only when no other session is mid-phase** — check PROGRESS.md's checkpoint and `git status` first; stage by explicit path, never `git add -A` (the 2026-07-12 parallel-session sweep is the scar).

Contents

01	Decisions I need from Bishan
02	The comment standard (and where your instinct was wrong)
03	The measurements (evidence, 2026-07-15)
04	The sacred list and the risk register (never strip, never reword)
05	The deletion table (LC1 executes <code>delete</code> rows after a fresh re-check)
06	Practice-efficiency fixes (all behavior-preserving; no guard weakened)
07	The phases
08	Adversarial pass
09	Appendix — Evidence files this plan produces

Decisions I need from Bishan

Defaults proceed; veto any row.

#	Decision	Default
0.1	The deletion list (Part 4's table).	Every row marked <code>delete</code> is deleted in LC1 after a fresh reference re-check. Rows marked <code>ask</code> wait for your word.
0.2	dummy/ (your 52 screenshots, 45 MB, untracked).	KEPT until Plan A's CC5 tags (they are its evidence), then deleted. Say the word to keep them longer.
0.3	README.md rewrite. It is the public landing page of a now-public repo and says "build not started" — affirmatively false.	LC1 rewrites it: what the product is, the document map, the honesty principles, current status. ~1 page, mechanical voice.
0.4	nav-timing.spec.ts appends samples on local runs , forcing the documented <code>git checkout --</code> ritual.	Gate the append on <code>process.env.CI</code> (local runs stop dirtying evidence). This changes how evidence is collected, so it is yours to veto.
0.5	The comment verdict below — targeted compression, not a total strip.	Part 1's counter-proposal is what this plan executes. If you want the total retroactive strip anyway, say so and LC3 becomes a 3-session grind with the risk register in Part 3 accepted item by item.

The comment standard (and where your instinct was wrong)

What you asked for: comment only the genuinely important; at most ONE line, TWO when truly necessary; delete restatement; keep the hard-won why. Applied retroactively everywhere.

What the measurement says: the repo is ~70,500 lines and ~21,650 of them ($\approx 30\%$) are comments or docstrings. But a restatement scan found only **5–10 true noise comments in the entire repo**. The volume is not noise — it is deliberate multi-line rationale, written under your own 2026-07-10 readability directive. The bloat is *verbose signal*, not junk. And a mass rewrite carries real, enumerated risk: six test files start with a machine-read `// @vitest-environment node` pragma; `eslint-disable` / `@ts-expect-error` / `# noqa` pragmas red the gate if touched; 28 trailing `// 2026-07-12` clock comments are REQUIRED by drift rule 21's design; several file headers are governance artifacts other guards cite (verified.ts, copy.ts, paper.mjs, brand-assets.mjs, the argued skip-lists inside check-drift.mjs); and **eleven drift rules match banned tokens without a comment exemption**, so *rewording* a comment in a policed file can red the build even though *deleting* whole comment lines never can.

The professional counter-proposal (this plan's law):

1. **New code, from the day LC1 tags:** your standard, verbatim — comment only where a reader genuinely benefits; ≤ 1 line, 2 only when truly necessary; never restate the code; the why (honesty rationale, bug archaeology, platform quirk) always earns its place.
2. **Retroactive work targets where the cost lives:** the top ~25 files by comment mass hold a fifth of all comment lines and are exactly the files every session reads (job_a.py, morning.ts, copy.ts, check-drift.mjs, page.tsx, publish.py, nightly.py, schema.prisma, the big specs...). LC3 compresses those — every why survives at ≤ 2 lines, essays become sentences, restatement dies. The long tail (400+ files whose comments are modest and rarely read) is NOT ground through in a dedicated pass.
3. **The boy-scout rule covers the tail:** any session that edits a file brings that file's comments to standard in the same commit. The tail converges without ever burning a session on it.
4. **The sacred list (Part 3) is never stripped or reworded**, in the top-25 pass or anywhere else.

This gets you the actual benefit — the hot files every session pays for become half their comment weight, and no new essay is ever written again — at a tenth of the risk and cost of a total strip.

The CLAUDE.md amendment (LC1 writes this, dated 2026-07-15, superseding the 2026-07-10 Readability point 3; points 1, 2 and 4 stand unchanged):

3. Comments are one line of why. (Amended 2026-07-15 — supersedes the "every non-trivial function gets a docstring" rule; the codebase it built taught us the failure mode.) Comment only where a reader would genuinely be lost without it. One line; two only when truly necessary. Never restate what the code says. The why always earns its place: an honesty-rule rationale, a hard-won bug, a platform quirk — keep those, tightly. Machine-read pragmas, the clocks rule's derived-date comments, and file-header laws cited by guards are load-bearing and are never stripped. When you edit a file, bring its comments to this standard in the same commit.

The measurements (evidence, 2026-07-15)

directory	files	total lines	comment+docstring	%
app/components	107	14,111	4,317	30.6%
app/lib	94	13,147	4,085	31.1%
app/app	53	6,239	1,666	26.7%
app/scripts	15	3,871	1,554	40.1%
app/e2e	28	5,890	2,193	37.2%
pipeline	129	22,511	6,596	29.3%
app/prisma	27	3,763	1,238	32.9%

Top files by comment mass (the LC3 worklist, refreshed at execution time by the same script): check-drift.mjs 478 · job_a.py 397 · copy.ts 353 · morning.ts 345 · narrate.py 327 · vrt.spec.ts 318 · nightly.py 288 · schema.prisma 286 · fixtures/news.mjs 275 · desk.spec.ts 253 · publish.py 235 · news.ts 203 · (desk)/page.tsx 199 · PipelinePanel.tsx 199 · hardening.spec.ts 189 — plus the next ten by the same measure, to ~25 files.

The measuring script (comment_stats.py — full-line comments via tokenizer, docstrings via ast, string literals never miscounted) is committed in LC1 under `scripts/` in the pipeline tree as `pipeline/scripts/comment_stats.py`, so the before/after claim in LC3's evidence file is reproducible, not folklore.

The sacred list and the risk register (never strip, never reword)

1. **Machine-read pragmas:** `// @vitest-environment node` (line 1 of six test files); `eslint-disable(-next-line)` (BrandMark.tsx, PipelinePanel.tsx, PipelineStrip.tsx); `@ts-expect-error` (RangeControl.test.tsx ×2); `# noqa: ...` (~20 pipeline sites). Stripping any of these reds lint, tsc, ruff, or flips a test's runtime.
2. **Clock-rule derived dates:** the 28 trailing `// 20xx-xx-xx` comments in seed.mjs, fixtures, and e2e — drift rule 21's own docstring ENCOURAGES them; they are the rule working.
3. **File-header laws and argued registers:** verified.ts (the E5 allow-list law), copy.ts (the Appendix J contract), fixtures/paper.mjs (rule 21 cites it as "the recording"), brand-assets.mjs's BRAND_FIELD provenance (rule 23 expects it), every argued entry inside check-drift.mjs's skip lists (P11: "argued in place" is a constitutional requirement), and the `run-name:` comment block in nightly-a.yml/nightly-b.yml (the dispatch-recovery law; test_workflow_dispatch.py guards the line it explains).
4. **The eleven comment-sensitive drift rules:** rules 3, 4, 5, 9, 10, 11, 13, 14, 18, 19, 20 have no comment exemption, and rules 6/24 exempt only whole-line comments. Consequence, and LC3's rule of thumb: **deleting a comment line is always drift-safe; writing or rewording one is not automatically safe.** Compression therefore prefers deleting lines to rewriting them, and any rewritten line in a policed file is checked against the rule list before commit.
5. **Tests that read source files** (the fonts display-pin from CC1, design-system checks): they match code tokens, not comments — verified — but LC3 re-runs the full unit suite per directory batch anyway, which is what actually proves it.

The deletion table (LC1 executes `delete` rows after a fresh re-check)

path	tracked	size	evidence	verdict
Screenshot 2026-07-12 at 6.20.46 PM.png (repo root; U+202F in name)	yes	1.3 MB	rode into a F5 feat commit; only reference is QUESTIONS-FOR-BISHANT.md:403, which itself says "delete it again and commit"	delete
docs/KICKOFF-PROMPT.md	yes	12 KB	the session-1 bootstrap; the build it bootstrapped is complete; zero references	delete
.github/workflows/.gitkeep	yes	0 B	directory holds five workflows	delete
pipeline/adapters/fixtures/.gitkeep	yes	0 B	directory holds nine fixture trees	delete
.vercel-auth-hash.tmp	no (disk)	60 B	.gitignore's own comment says "delete after deploy" (2026-07-10)	delete
root node_modules/ (pixelmatch+pngjs, no root package.json)	no (disk)	small	residue of a hand-rolled VRT diff — replaced by the committed tool in LC1 (Part 5.4)	delete
.DS_Store x11 (disk only; none tracked)	no	~100 KB	OS junk	delete
dummy/ (52 PNGs)	no	45 MB	Plan A's CC1–CC5 evidence	ask (default: delete after CC5 tags — Part 0.2)
README.md	yes	—	states "build not started"; omits five plan documents; the repo is public	rewrite (Part 0.3)
GATE-EFFICIENCY-ANALYSIS.md	yes	—	cited by GATE-EFFICIENCY-PLAN.md ("read it in full"), by four ci.yml comments, and by two gate-evidence files	keep
FigmaDesignRef/ (1.9 MB Vite app)	yes	1.9 MB	design provenance, cited once by UI-REDESIGN-PLAN.md:22; never built or imported	keep (flagged; if ever pruned, the citing line gets a dated note — and note its own CLAUDE.md/AGENTS.md can confuse tree-walking agent tooling)

path	tracked	size	evidence	verdict
news/ (3 reference PNGs)	yes	3.3 MB	cited by NEWS-AND-CONTROL-PLAN.md:806 as the commissioned reference set	keep (retained record)
UI-LIBRARY-EVALUATION.md + its PDF + HTML	untracked	—	NEXT-SESSION-PROMPT.md:19: "a finished research deliverable, deliberately uncommitted — leave them"	keep

Swept and found clean (no candidates): app/public (every asset referenced), assets/ (brand master + fonts + licenses, all consumed), prisma/fixtures (all imported by seed), e2e helpers (all imported), app/components and app/lib (zero orphan modules), app/scripts (all 13 reachable), pipeline/scripts (recorder convention per the new-provider-adapter skill — but see 5.6).

Practice-efficiency fixes (all behavior-preserving; no guard weakened)

1. **One e2e session helper.** `signIn(page)` is byte-identical in 15 spec files, inlined 5 more times in `desk.spec.ts`, and every spec repeats the USER/PASSWORD header. LC2 creates `app/e2e/session.ts` exporting `USER`, `PASSWORD`, `signIn` and rewrites all 24 specs to import it. Pure extraction; `vrt`'s own `shoot()` mouse-parking stays untouched where it is.
2. **`waitForLayout` becomes importable.** `CLAUDE.md` commands every box-measuring spec to call it, but it is a private function inside `grid.spec.ts` — the law currently requires copy-paste. LC2 moves it into the shared helper, exported. This *strengthens* a standing rule.
3. **Guard-script plumbing dedupes.** The session-cookie mint exists three times (check-nav, check-live, lighthouse-check) and manifest parsing twice each (routes manifest: check-nav + check-bundles; build manifest: check-routes + check-bundles). LC2 extracts `app/scripts/lib/session-cookie.mjs` and `app/scripts/lib/manifest.mjs`. Every assertion, threshold and rule stays inside its guard file. Proof of preservation: capture each guard's full stdout on the same built app before and after; diff must be empty.
4. **The VRT candidate-diff tool gets committed once.** The standing law says "diff every candidate, decode both, count differing pixels" and no tool exists — it has been re-derived by hand at least once (the orphaned root `pixelmatch/pngjs` prove it). LC1 commits `app/scripts/vrt-diff.mjs` (~40 lines: two dirs in, per-file changed-pixel counts out, deps in app devDependencies) and cites it from `.claude/skills/vrt-update/SKILL.md`. A mandatory ritual becomes cheap instead of re-derived — the opposite of weakening it.
5. **One false sentence in `CLAUDE.md`.** Endgame step 3 says the rehearsal runs "all three shards"; the oracle has four legs since PD3 (`ci.yml`'s own matrix). LC1 corrects the word. (The constitution is otherwise out of this plan's scope, deliberately — it is the user's document.)
6. **`record-fixtures.yml` drift.** The new-provider-adapter skill says every recorder joins that workflow's steps; only 3 of 10 are steps. LC2 reconciles (add the seven, or amend the skill to say recorders run locally — executor judgment, one line in `DECISIONS.md` either way).
7. **nav-timing local-append ritual** — Part 0.4's decision; if approved, the append gates on CI and the documented checkout ritual is deleted from `CLAUDE.md`'s e2e notes in the same commit.

The phases

Standing gate per phase: the Endgame (CLAUDE.md) — local gate → push → confirm branch CI → rehearse when pixels or e2e are touched → tag `lc-N` by SHA → one docs commit. FIRST STEP of LC1: wire the `lc-*` tag family into ci.yml's tag filter (`test_ci_tag_families.py` names the expectation; push before any dispatch).

LC1 — The standard, the deletions, the tool. Tag `lc-1`. CLAUDE.md amendment (Part 1's text verbatim) + the "three shards"→"four legs" correction + commit `comment_stats.py` + commit `virt-diff.mjs` + skill citation + execute Part 4's `delete` rows (fresh `grep -r` per row first; a row that has gained a reference since 2026-07-15 is skipped and logged) + README rewrite (0.3). No pixels change; no rehearsal needed (docs+scripts push runs the normal branch CI). Evidence file: `docs/lean-evidence/lc1.md` with the deletion receipts (path, size, ref recheck output).

LC2 — Consolidation. Tag `lc-2`. Part 5.1–5.3 + 5.6 (+ 5.7 if approved). The e2e rewrite touches 24 specs — mechanical, but it IS the browser suite, so the gate includes a full rehearsal (`gh workflow run ci.yml -f job=e2e` on the exact SHA) and the guard-output before/after diffs from 5.3 land in the evidence file. VRT: no pixels should move; if any leg reds, that is a finding, not a baseline to refresh.

LC3 — The hot-file compression. Tag `lc-3`. The top-25 list (Part 2, refreshed), under Part 1's law and Part 3's register. Mechanics: work in four batches (app/lib+app — components — e2e+scripts — pipeline+prisma); after each batch run the comment-only PROVER plus that tree's tests before moving on. **The prover** (committed as `pipeline/scripts/comment_prover.py`, used for every batch):

- TS/TSX/MJS: tokenize old and new with the TypeScript scanner; the non-comment token streams must be identical. Any difference = the batch is rejected, not explained.
- Python: `ast.dump` equality after normalizing docstrings out of both trees, PLUS an assertion that the only changed string constants ARE docstrings. (Docstring compression legitimately edits the AST; nothing else may.)
- prisma / css / yaml (no tokenizer): whole-line-deletion-only — every diff hunk in these files must consist of removed full-line comments or shortened comment lines; the prover checks the hunks mechanically and anything else is rejected.

Gate: full local gate + `uv run pytest` + one rehearsal at the end (comments do not render, but the suite proving that is the point) + before/after `comment_stats.py` table in the evidence file (`docs/lean-evidence/lc3.md`). Expected outcome, honestly stated: roughly half the comment mass in those 25 files ($\approx 4\text{--}6\text{k}$ lines) with every why intact; the repo-wide percentage moves modestly — the recurring token cost of the hot files is the prize, not the global number.

Adversarial pass

1. **The prover passes but a pragma died.** Pragmas are comments — token-stream equality would still pass. Counter: the prover explicitly greps the sacred patterns (Part 3.1) in old vs new and requires identical counts per file; and the gate's lint/tsc/ruff/vitest would red anyway.
2. **A reworded comment trips a drift rule.** Rule of thumb enforced in review: prefer deletion; any rewritten line in app/** runs check:drift before commit (it is in the local gate regardless).
3. **Another session is mid-phase during LC's wide touches.** The sequencing law in the cover: check PROGRESS.md + git status; explicit-path staging only; if in doubt, LC waits — hygiene never races product work.
4. **Deleting KICKOFF-PROMPT orphans a citation someday.** The fresh per-row re-check at execution time is the counter; a row that gained a reference is skipped and logged, never forced.
5. **The e2e consolidation subtly changes login timing.** signIn is byte-identical today — the import changes nothing but the byte count. The rehearsal (full four-leg oracle) is the proof, and any flake there gets read, not re-run blind (the G2 lesson).
6. **Compression lobotomizes a war story the next session needed.** The sacred list keeps the named laws verbatim, and compression keeps every why at ≤ 2 lines — shorter, not gone. The DECISIONS/LESSONS files (untouched by this plan) remain the long-form memory. If a compressed why proves too thin in practice, restoring prose is a one-file `git log -p` away — nothing is lost from history.

Appendix — Evidence files this plan produces

docs/lean-evidence/lc1.md (deletion receipts, README before/after, tool listings) · lc2.md (guard-output identity diffs, spec-count table, rehearsal run id) · lc3.md (comment_stats before/ after per file, prover output per batch, rehearsal run id). Each ends with the standard gate-size line, per the Endgame's rule 8.
